

The Power of 11: Adapting the Power of Ten for Safety-Critical Python

The Paradigm Shift in Safety-Critical Computing

The development of safety-critical software has historically relied upon compiled, statically typed languages including C and Ada. This reliance is largely due to the deterministic memory management, predictable execution models, and mature static analysis tooling associated with these languages.¹ In 2006, the Jet Propulsion Laboratory (JPL) formalized the "Power of Ten" rules to constrain C programming into a highly analyzable, predictable subset suitable for mission-critical deployments where human lives and high-value assets are at risk.¹ These rules prioritize structural simplicity, bounded control flow, and strict memory management to enable comprehensive static analysis, operating under the philosophy that complex code is inherently untestable and therefore unsafe.¹

However, the modern software engineering landscape increasingly relies on Python. Its rapid prototyping capabilities, extensive standard library, and unmatched dominance in artificial intelligence, data science, and machine learning have made it ubiquitous.³ Adapting Python for safety-critical systems presents a fundamental architectural friction. Python is an interpreted, dynamically typed language with automatic garbage collection, a Global Interpreter Lock (GIL), and inherent memory allocation overhead.³ Unpredictable garbage collection pauses and dynamic heap allocations violate the core tenets of hard real-time determinism, where a microsecond delay can result in catastrophic system failure.⁴ Furthermore, the lack of compile-time error checking means that type mismatches and reference errors often only manifest at runtime.³

To bridge this operational gap, a rigorous new paradigm is required. This report translates the C-centric Power of Ten rules into Pythonic equivalents, leveraging structural constraints to enforce determinism. Furthermore, it outlines how these rules are systematically enforced through high-performance, Rust-based toolchains (specifically uv and Ruff), standardizes documentation practices for long-term maintainability, and integrates comprehensive AI security controls to defend against emerging algorithmic and generative threats.

Architectural Scope: The Tiered Deployment Profiles

Modern safety-critical systems are rarely monolithic. A modern aerospace and automotive system relies on a spectrum of compute layers—ranging from AI-driven mission computers down to bare-metal actuator controllers.

Because Python's execution environment heavily depends on the underlying hardware, enforcing a single set of constraints across all layers creates technical contradictions (specifically mandating NumPy on a microcontroller that cannot support C-extensions).

Therefore, the "Power of 11" standard operates on a Tiered Architecture, explicitly dividing the system into two strict deployment profiles. Every Python module must explicitly declare which profile it targets in its docstring.

Profile A: High-Level Mission Compute (The "Edge" Profile)

This profile governs systems with full operating systems (Linux, RTOS) and significant compute resources. It is designed for high-level telemetry processing, data routing, and Artificial Intelligence execution.

- **Target Runtime:** CPython 3.11+
- **Target Hardware:** Mission computers, ground stations, NVIDIA Jetson, and Raspberry Pi Compute Modules.
- **Core Characteristics:**
 - Permitted to use heavily vetted C-extension libraries (NumPy, Pydantic).
 - Bound by the AI-Critical constraints (gVisor sandboxing, safetensors weights, NeMo-Guardrails).
 - Hard real-time execution is not guaranteed; relies on soft real-time bounded latency.

Profile B: Low-Level Embedded Control (The "Embedded" Profile)

This profile governs code executing directly on hardware and tightly constrained microcontrollers. It is designed for hard real-time sensor fusion, hardware interrupts, and actuator control loops.

- **Target Runtime:** MicroPython (and CircuitPython)
- **Target Hardware:** Bare-metal microcontrollers (specifically ARM Cortex-M and ESP32), Electronic Control Units (ECUs), and flight controller loops.
- **Core Characteristics:**
 - Zero C-Extension Policy: Pure Python and native MicroPython modules only.
 - Explicit manual control over Garbage Collection (gc.disable()) during critical loops).
 - Hard real-time determinism is paramount.

The Pythonic Power of 11: Architectural Translation

The original Power of Ten rules target the specific idiosyncrasies and vulnerabilities of the C programming language, specifically raw pointer arithmetic, manual memory allocation, and the obfuscation capabilities of the C preprocessor.¹ Applying these principles to Python requires translating low-level memory and compilation concepts into high-level structural, architectural, and dynamic constraints.

Original C Rule (NASA/JPL)	Pythonic Adaptation for Safety-Critical Code	Primary Enforcement Mechanism

1. Avoid complex flow (goto, recursion)	Prohibit recursion, deep nested generators, and exception-based control flow loops.	Ruff (C901, PLR1702)
2. Fixed upper bounds for all loops	Avoid unbounded while loops; mandate bounded iterators (range(), itertools.islice).	Static Analysis / Peer Review
3. No dynamic memory after initialization	Pre-allocate collections; enforce __slots__; manage GC explicitly (Profile dependent).	Architectural Design, gc module
4. Limit function length (<60 lines)	Limit functions to 50 statements to reduce cognitive and cyclomatic complexity.	Ruff (PLR0915)
5. Minimum two assertions per function	Utilize structural validation and explicit raise over assert (Profile dependent).	Ruff (S101), Pydantic
6. Smallest possible scope for data	Avoid global and nonlocal; enforce immutable data passing.	Ruff (PLW0603)
7. Check return values and parameters	Enforce strict static typing; utilize Typels for boundary validation.	Pyright (Strict), Typels

8. Limit preprocessor use	Prohibit dynamic metaprogramming (eval, exec, globals(), setattr).	Ruff (S102, S307, B010)
9. Limit pointer dereferencing	Restrict reference aliasing; mandate deep immutable data structures.	Architectural Design
10. Compile with all warnings enabled	Zero-tolerance linting with Ruff; deep taint analysis with Semgrep.	CI/CD Pipeline, Semgrep
11. Mandatory Dependency Vetting	Pin dependencies to hashes, vet for security vulnerabilities, and prohibit floating versions.	uv.lock, pip-audit

Rule 1: Restricting Control Flow and Bounding Execution

The first rule of the Power of Ten mandates that all code must be restricted to very simple control flow constructs, explicitly prohibiting goto statements, setjmp and longjmp constructs, and direct and indirect recursion.¹ The rationale is that simpler control flow translates into stronger capabilities for verification and results in improved code clarity.¹ The prohibition of recursion is critical because it guarantees an acyclic function call graph, allowing static code analyzers to prove that all executions are bounded and preventing stack overflow vulnerabilities.¹

In Python, the goto statement does not exist natively, eliminating that specific threat vector. However, Python's flexibility introduces other complex control flow mechanisms that must be aggressively constrained in safety-critical environments. Recursion must be strictly prohibited, just as it is in C.⁵ Python's call stack is finite and managed dynamically; recursive exhaustion leads to unrecoverable RecursionError crashes that halt the interpreter. By mandating purely iterative processes, the call graph remains acyclic and predictable. Static analyzers can then accurately model state transitions. Furthermore, exception handling must not be used as a primary control flow mechanism and "hidden gotos" (specifically using StopIteration manually and breaking loops via try-except blocks), as this obfuscates the execution path and degrades

performance; exceptions must be strictly reserved for signaling actual errors.⁶

Rule 2: Establishing Fixed Loop Upper-Bounds

The second rule dictates that all loops must have a fixed, preset upper bound that can be trivially proven statically by a checking tool.¹ Combined with the prohibition of recursion, fixed loop bounds prevent "runaway code" and ensure system termination and bounded execution times.¹

Python heavily utilizes iterators and generators, and developers should strictly prefer for loops over finite iterables⁶ (specifically for item in predefined_list). However, unbounded while True: constructs, frequently used in asynchronous event loops and daemon processes, introduce severe liveness risks. In safety-critical Python, developers must avoid unbounded loops. When polling hardware and waiting for asynchronous network operations, strict timeouts must be enforced. If a loop relies on a dynamic condition, safety-critical guidelines mandate the inclusion of a hard iteration limit. Developers must utilize itertools.islice to forcefully truncate potentially infinite generators and implement explicit iteration counters within while loops⁶ to prevent infinite execution. This ensures that even in the presence of logical errors, the loop will eventually terminate, returning control to the system scheduler.

Tiered Architecture Adaptation:

- **For Profile A (CPython):** May raise a TimeoutError exception when a loop bound is exceeded.
- **For Profile B (MicroPython):** Must break the loop and immediately return an explicitly typed Error Result object (specifically (False, None, ERR_LOOP_TIMEOUT)). Exceptions for expected control flow—including loop bounds—are strictly forbidden in Profile B.

Rule 3: Memory Determinism and Pre-Allocation

Rule 3 prohibits dynamic memory allocation after the software has been initialized.¹ In C, memory allocators including malloc and free exhibit unpredictable behavior, lead to heap fragmentation, and cause catastrophic errors including memory leaks, use-after-free bugs, and allocation failures due to insufficient physical memory.¹

Applying this rule to Python requires a paradigm shift, as Python fundamentally abstracts memory management away from the developer. Python relies on a private heap containing all objects and data structures, managed by object-specific allocators and an automatic garbage collector based on reference counting and cyclic isolation.⁷ The garbage collector can pause execution unpredictably to reclaim memory, a non-deterministic behavior entirely unacceptable in hard real-time environments (including aerospace flight controllers and automotive braking systems) where a delay of a few milliseconds can be fatal.⁴ Furthermore, dynamic typing and object metadata mean that a simple integer in Python consumes significantly more memory than a C integer, rapidly depleting the constrained RAM budgets of embedded systems.⁴

To achieve pseudo-static memory allocation in Python, developers must leverage pre-allocation strategies and deep structural constraints. Collections whose ultimate size is known must be pre-allocated at startup. Developers must utilize NumPy's `np.zeros()`, `bytearray(n)`, and `collections.deque(maxlen=n)` to allocate contiguous, fixed-size buffers and bounded queues in memory.⁶ This avoids the fragmented, pointer-chasing overhead of dynamically appending to a standard Python list and dictionary without bounds during runtime.⁶ If an allocation could potentially fail, the code must be prepared to handle `MemoryError` exceptions explicitly.⁶

Equally critical is the adoption of the `__slots__` attribute in class definitions. By default, Python objects store their instance variables in a dynamic dictionary (`__dict__`), which carries significant memory overhead and allows attributes to be arbitrarily added and removed at runtime.⁹ Declaring `__slots__ = ['x', 'y', 'z']` instructs the Python interpreter to suppress the creation of the `__dict__` and instead allocate a fixed-size array for the specified attributes.¹⁰ This achieves two vital safety goals: it reduces the memory footprint by up to 50% per instance, preventing out-of-memory errors in constrained environments, and it statically locks the object's schema, preventing the accidental introduction of unexpected state mutations at runtime.⁹

Because Python's interpreter overhead and dynamic frame management make static stack analysis considerably more difficult than in C, relying purely on structural prevention is insufficient. Safety-critical Python testing protocols must therefore incorporate mandatory runtime monitoring of memory high-water marks. Utilizing profiling tools and the native resource module, automated test suites must continuously measure heap and stack allocations during execution.¹¹ This guarantees that the application's maximum memory footprint never exceeds the absolute physical limits of the target hardware.

Tiered Architecture Adaptation:

- **For Profile A (CPython):** Developers must use NumPy to pre-allocate memory blocks for large data streams (specifically `np.zeros(1000)`). The use of dynamic Python lists (```) appended in a loop is strictly forbidden for data buffering.
- **For Profile B (MicroPython):** NumPy is forbidden. Developers must pre-allocate contiguous memory using Python's native array module (specifically `buffer = array.array('f', [0.0] * 100)`). Furthermore, developers must explicitly disable the garbage collector (`gc.disable()`) before entering a hard real-time control loop, running `gc.collect()` manually only during predefined, safe idle windows.

Rule 4: Limiting Function Length and Complexity

The Power of Ten mandates that functions should not exceed the length of a single sheet of paper (approximately 60 lines of code).¹ Excessively long functions are often an indicator of poorly structured code, making the logical unit difficult to understand, test, and verify as a cohesive unit.¹

In Python, this metric is strictly enforceable using static analysis. The Ruff linter provides the PLR0915 rule (Too many statements), which flags functions and methods containing an excessive number of statements, defaulting to a limit of 50.¹² By enforcing this rule, organizations compel developers to refactor complex logic into smaller, independently testable helper functions.¹³ This limitation operates in tandem with cyclomatic complexity checks. Ruff implements the McCabe complexity metric via the C901 rule, which assesses the complexity of the control flow graph.¹² A strict upper bound on McCabe complexity (mandating a strict maximum complexity of 10) ensures that the number of linearly independent paths through a function remains low enough to achieve 100% branch coverage during unit testing without combinatorial explosion.

Rule 5: Structural Validation Over Native Assertions

Rule 5 dictates that the code should maintain a minimum average of two assertions per function to check for anomalous conditions that should never happen in real-life executions.¹ In C, assertions are side-effect-free Boolean tests that trigger recovery actions and error logs upon failure.¹

A direct translation of this rule to Python introduces a severe security vulnerability. In Python, the native `assert` statement is designed purely for internal debugging. If the Python interpreter is executed with optimization flags (specifically `python -O`), all `assert` statements are entirely stripped from the compiled bytecode.¹² Consequently, relying on `assert` for runtime safety validation, input sanitization, and boundary checking means those critical defenses will silently disappear in a production deployment.¹⁵

The Pythonic equivalent mandates the strict prohibition of `assert` in application logic. Ruff's S101 (flake8-bandit) rule explicitly detects and flags the use of `assert`, preventing this oversight.¹² Instead of assertions, developers must use explicit, un-optimizable validation layers. This involves utilizing explicit `if not condition: raise CustomError` constructs to halt execution when invariants are violated.

Tiered Architecture Adaptation:

- **For Profile A (CPython):** Developers must use structural validation frameworks, specifically Pydantic, to define strict schemas at the boundaries of the application. Pydantic guarantees data integrity, type coercion, and schema enforcement before the data ever enters the core business logic.
- **For Profile B (MicroPython):** Pydantic is prohibited because it relies on forbidden metaprogramming and consumes excessive resources. Developers must use explicit, manual guard clauses (specifically `if not condition: return ErrorState`) to reject invalid state.

Rule 6: Minimizing Data Scope and Mitigating Global State

Data objects must be declared at the smallest possible level of scope.¹ This basic principle of

data-hiding ensures that if an object is not in scope, its value cannot be accidentally referenced and maliciously corrupted.¹ It also drastically simplifies fault diagnosis by limiting the number of statements where a variable's value could have been mutated.¹

Python's scoping rules (LEGB: Local, Enclosing, Global, Built-in) lack the granular block-level scoping found in C and C++. Variables defined within a for loop and if block leak into the broader function scope. To adhere to this rule, developers must aggressively restrict state visibility. The use of the global and nonlocal keywords must be entirely forbidden, as they introduce hidden dependencies and untrackable state mutations across function boundaries. State must be passed explicitly through function parameters (dependency injection). Furthermore, the reuse of variables for incompatible purposes must be prohibited to prevent type confusion and diagnostic obfuscation. Developers must also strictly avoid mutable default arguments (specifically `def f(acc=)`) because their state persists across function calls and introduces hidden global mutability.⁶ Ruff enforces these principles through rules that flag unused variables, shadowed built-ins, and complex variable reuse.

Rule 7: Rigorous Parameter and Return Value Verification

Calling functions must meticulously check the return values of non-void functions, and the validity of parameters must be checked inside every function.¹ This prevents the silent propagation of errors up the call chain, a common source of catastrophic failures when handling standard library interactions and system I/O.¹

The original NASA JPL document notes that standard C libraries famously violate these safety rules (specifically executing `strlen(0)`) and warns developers to remain extremely wary.¹ Given Python's heavy reliance on external pip packages and its extensive standard library, this warning must be strictly formalized. A stronger Pythonic adaptation dictates that all interactions with the standard library and third-party packages be treated as untrusted boundaries. Any permitted third-party library must undergo a rigorous vetting process, requiring high proven test coverage and successful compliance with the same Ruff (Bandit) security checks mandated for internal code.

In modern Python industry standards, internal parameter verification is fulfilled through comprehensive static type hinting (PEP 484) combined with the automated type checker **Pyright**.¹⁶ Because manual review of large codebases is computationally and humanly infeasible,¹ verification must be checked mechanically. Crucially, to prevent a single point of failure in this static analysis pipeline, Pyright must be explicitly configured in its most pedantic mode (strict level). This configuration must absolutely prohibit the use of the Any type. In Python, Any acts as a static analysis "black hole"—it silently disables type checking for that object and its downstream derivatives, creating hidden logic flaws that bypass mechanical verification entirely.

Explicit Type Narrowing for Untrusted Data: When handling external and untrusted data, basic type hints are insufficient. The safety-critical standard explicitly requires the use of rigorous type narrowing constructs to track data boundaries. Developers must utilize functions

annotated with Typels (introduced in PEP 742 for Python 3.13+). Typels is the strictly mandated paradigm because it mathematically guarantees safe type narrowing in both the positive (if) and negative (else) branches of control flow, ensuring that static analyzers accurately model the sanitized data state across the application's execution graph.

Tiered Architecture Adaptation:

- **For Profile A (CPython):** Bounded try/except blocks are permitted at the architectural boundaries (specifically catching network timeouts), provided the exception does not dictate the core business logic. Furthermore, when a function can return an error state, Python developers must leverage explicit Union types (mandating modern Python 3.10+ X | Y syntax).¹⁷ Static type checkers then force the caller to explicitly handle the None case and the error variant before accessing the underlying data.
- **For Profile B (MicroPython):** Exceptions for expected control flow are strictly forbidden. Raising exceptions breaks the acyclic call graph and causes unpredictable stack unwinding. Functions must instead return explicit Result objects (specifically a NamedTuple containing (success_bool, value, error_code)). To prevent deep if/elif chains that violate complexity limits, Result objects must be explicitly unpacked using Python 3.10+ Structural Pattern Matching (match / case) to maintain a flat, highly readable control flow graph. raise is reserved exclusively for catastrophic hardware panics that require an immediate system halt and reboot.

Rule 8: Eliminating Metaprogramming and Dynamic Execution

Rule 8 limits the C preprocessor to header file inclusion and simple macro definitions, explicitly forbidding token pasting, recursive macro calls, and complex conditional compilation.¹ The rationale is that the preprocessor is a powerful obfuscation tool that can destroy code clarity, befuddle static analyzers, and exponentially increase the required testing effort by creating hidden alternate code paths.¹

Python does not possess a preprocessor, but it features dynamic metaprogramming capabilities that are vastly more dangerous to static analysis. Features including eval(), exec(), globals(), locals(), dynamic attribute manipulation (getattr and setattr with variable keys), and monkeypatching actively rewrite the execution environment at runtime.¹² These features destroy the ability of static analysis tools to understand the code's behavior and verify data types.¹⁷

Therefore, dynamic execution and metaprogramming must be strictly forbidden in safety-critical Python. Code must be explicitly defined and statically discoverable. The Ruff static analyzer provides automated enforcement against these dynamic anti-patterns via its Bandit rule set, specifically S102 (detecting exec) and S307 (detecting eval, mandating the safe ast.literal_eval).¹² Additionally, unchecked subprocess calls and unsafe deserialization patterns are entirely banned due to their unpredictable and dangerous dynamic behaviors.⁶

Tiered Architecture Adaptation:

- **For Profile A (CPython):** Decorators are permitted, provided they utilize typing.ParamSpec so that Pyright can mathematically verify the inputs and outputs of the wrapped function.
- **For Profile B (MicroPython):** Custom decorators are strictly forbidden to ensure the code can be easily compiled into immutable ROM. Dynamic module importing (using `__import__` and importing inside a function block) is strictly forbidden; all imports must be declared statically at the top of the file.

Rule 9: Restricting Reference Aliasing and Enforcing Immutability

The ninth rule restricts the use of pointers to no more than one level of dereferencing and completely prohibits function pointers, as deep pointer indirection makes it exceptionally difficult to analyze the flow of data.¹

While Python abstracts memory addresses away and lacks explicit pointers, its memory model relies entirely on pass-by-object-reference.¹⁸ Every variable assignment is essentially a reference to an object in the heap. Passing a mutable object (specifically list and dict) into a function allows that function to silently mutate the caller's state.¹⁸ This creates hidden side effects, deep state indirection, and temporal coupling that directly violates the intent of Rule 9.

The Pythonic equivalent requires the aggressive use of immutable data structures to prevent state corruption. Developers must favor tuple over list, frozenset over set, and leverage data classes defined with `@dataclass(frozen=True)`.¹⁹ Tuples and frozen dataclasses must only contain other immutable types; nesting mutable objects inside immutable containers is strictly forbidden. State must be managed exclusively through pure functions that return new instances; in-place mutation and deep copying are strictly prohibited. This guarantees referential transparency, ensures predictable data flow, and limits the blast radius of localized logical errors. Furthermore, higher-order functions (passing functions as arguments) must be minimized in critical execution paths, as they obscure the control flow graph much like C function pointers do.¹

Rule 10: Pedantic Compilation and Continuous Zero-Tolerance Analysis

The final rule dictates that all code must be compiled from the first day of development with all warnings enabled at the most pedantic settings, and it must pass daily checks by state-of-the-art static analyzers with zero warnings.¹

In the interpreted Python ecosystem, this translates to utilizing a highly deterministic, zero-tolerance static analysis pipeline that operates continuously in the CI/CD environment. A fully compliant project integrates `uv run ruff check` continuously. Adherence to a zero-warnings policy is absolute. The use of `# noqa: overrides` is strictly prohibited; if a rule violation occurs, the code must be refactored until it complies.¹

Beyond AST Linters (Taint Analysis): While Ruff provides exceptional speed and

comprehensive formatting, its architectural scope is fundamentally limited to acting as an Abstract Syntax Tree (AST) linter. For safety-critical software, AST linting alone is inadequate because it cannot track vulnerabilities that weave through deep execution paths. Therefore, the analysis pipeline must also mandate the use of the **Semgrep** cross-file control-flow and taint-analysis tool. Taint analysis systematically tracks the flow of untrusted data from an entry point through the entire architecture, catching complex security injections that local linters inherently miss.

The combination of uv for reproducible environments, Ruff for pedantic structural constraints, and Semgrep for deep data-flow analysis forms the bedrock of this compliance requirement.

Rule 11: Mandatory Dependency Vetting and Supply Chain Integrity

While the original NASA rules focused on the isolation of the C source code, modern Python development is inseparable from its ecosystem. In a safety-critical Python environment, a vulnerability in a third-party package is functionally equivalent to a logic error in your own code.

The Rule

All external dependencies must be pinned to specific hashes, vetted for security vulnerabilities, and mirrored in a private, immutable artifact repository. No "floating" versions and direct connections to public indices (PyPI) are permitted during build and deployment.

Rationale

In C, dependencies are often static and limited to a few stable libraries. In Python, a single pip install can pull in dozens of transitive dependencies. If any of these are compromised via typosquatting attacks and protestware updates, the safety of the entire system is invalidated. Furthermore, the dynamic nature of Python allows libraries to modify global state and monkey-patch built-in functions, creating non-deterministic behavior that static analysis of your code cannot catch.

Implementation Requirements

- **Cryptographic Pinning (The "Lockfile" Requirement):** You must use a tool that generates a content-addressable lockfile (mandating uv.lock).²⁰
 - **Constraint:** Every dependency must have a SHA-256 hash. This ensures that the code running in production is bit-for-bit identical to the code vetted in testing.
- **Automated Vulnerability Scanning:** All dependencies must be continuously screened against the CVE (Common Vulnerabilities and Exposures) and GitHub Advisory databases.
 - **Tooling:** Integration of **pip-audit** into the CI/CD pipeline is mandatory. Any "High" and "Critical" vulnerability must trigger an immediate build failure.
- **The "Minimalist" Doctrine:** Before adding a library, developers must justify why the functionality cannot be implemented using the Python Standard Library.
 - **Constraint:** Avoid "micro-packages" (specifically libraries that perform a single, trivial task like left-pad). These increase the attack surface without providing

significant architectural value.

- **License and Provenance Auditing:** Dependencies must be audited for restrictive licenses (specifically the GPL in a proprietary safety-critical context) and "bus factor" risks.
 - **Requirement:** Check the maintenance health of the package. If a library has not been updated in over 24 months and has fewer than two active maintainers, it is considered a "Safety Risk" and must be rejected and replaced with compliant alternatives.
- **Runtime Sandboxing:** For high-integrity systems, use **strict containerization** to limit the system calls a Python process can make. This prevents a compromised third-party library from accessing the network and sensitive file system paths.

Example of a Rule 11 Violation:

Python

```
# VIOLATION: Using a floating version in requirements.txt
pandas>=2.0.0
```

```
# COMPLIANT: Pinned with hash (via uv)
pandas==2.2.1 \
--hash=sha256:7f...
```

By enforcing Rule 11, you extend NASA's principle of "Analyzability" beyond your own script and into the entire execution environment, ensuring that the "Power of Ten" isn't undermined by an invisible piece of code five layers deep in your dependency tree.

Inter-Profile Communication: The Airgap Rule

Profile A and Profile B code must never share memory and run in the same process. They must be treated as completely isolated systems. Communication between an Edge (Profile A) node and an Embedded (Profile B) node must occur over a serialized hardware bus (specifically CAN bus, UART, and SPI) using a mathematically strictly defined schema, specifically Protocol Buffers (protobuf) and explicitly packed binary structs.

Modern Python Toolchain: Reliability Through Determinism

Safety-critical software cannot tolerate the "works on my machine" anti-pattern.²¹ Dependency drift—where upstream open-source libraries introduce unexpected breaking changes, hidden transitive dependencies, and security vulnerabilities—can instantly compromise a previously

stable system.²¹ Traditional Python packaging tools historically lacked strict, universal lockfile enforcement by default, leading to fragmented and irreproducible environments.²¹

The industry standard for mitigating these risks relies on a modern, Rust-powered toolchain designed for absolute determinism and extreme performance: uv and Ruff.

Environmental Reproducibility with uv

The uv package manager, written entirely in Rust, serves as a high-performance, drop-in replacement for legacy tools including pip, pip-tools, poetry, and pyenv.²² Its primary architectural advantage in a safety-critical context is its rigorous enforcement of environment reproducibility through universal lockfiles.²²

When developers initialize a project and add dependencies, uv computes a comprehensive dependency resolution graph in milliseconds and generates a uv.lock file.²⁰ This file contains a complete, frozen list of every direct and transitive package, alongside their exact versions and cryptographic hashes.²⁰ This ensures that malicious tampering of upstream packages and accidental version bumps are caught immediately.

For safety-critical continuous integration and deployment (CI/CD) pipelines, the execution of uv sync --frozen and uv sync --locked is non-negotiable.²⁰ These commands force the installer to strictly adhere to the lockfile without attempting to re-resolve dependencies. If a developer alters the pyproject.toml without properly regenerating and committing the lockfile, the CI build will deterministically fail, preventing untested dependency matrices from entering the production environment.²⁰ Furthermore, uv allows for explicit Python interpreter version pinning (specifically uv python pin 3.12), which eliminates subtle behavioral variations caused by differing host implementations of the CPython runtime.²⁵

Zero-Tolerance Static Analysis with Ruff

Ruff is an exceptionally fast Python linter and code formatter, also written in Rust, designed to replace a fragmented ecosystem of specialized tools including Flake8, Pylint, Bandit, isort, and pyupgrade.²³ Because it executes natively without the overhead of the Python GIL, it analyzes massive codebases in milliseconds, allowing it to be run constantly as a pre-commit hook without degrading developer velocity.²³

For safety-critical compliance equivalent to Rule 10, Ruff must be configured via pyproject.toml and ruff.toml to enforce a highly aggressive, pedantic subset of over 800 available rules.²³ Key rule sets include:

1. **Complexity and Refactoring (C90 and PLR):** Ruff natively re-implements the McCabe complexity algorithm (C901) to limit control flow graph complexity.¹² Furthermore, the Pylint Refactor (PLR) suite actively monitors code volume, throwing errors for functions exceeding allowed limits for arguments (PLR0913), local variables (PLR0914), branches (PLR0912), and nested blocks (PLR1702).¹²

2. **Security and Defensive Coding (S - Bandit):** The flake8-bandit rules automatically detect insecure coding patterns that lead to exploitation. This includes flagging the dangerous use of assert in production (S101), identifying hardcoded cryptographic keys and passwords (S105, S106), preventing unsafe deserialization through Python's pickle module (S301), and warning against network requests lacking explicit timeouts (S113).¹²
3. **Exception Handling Safety (TRY):** The tryceratops rules ensure that error handling does not mask underlying failures. Rules including TRY002 prevent the raising of generic Exception classes, forcing developers to build a granular error taxonomy. TRY400 mandates the use of logging.exception to ensure complete stack traces are securely captured during catastrophic failure.¹²
4. **Concurrency and Asynchronous Safety (ASYNC):** To guarantee execution determinism, the native threading module is **strictly forbidden**. Python's Global Interpreter Lock (GIL) makes thread context-switching fundamentally unpredictable, violating hard real-time constraints. All concurrency must be achieved through strictly bounded asyncio event loops for I/O-bound tasks and isolated multiprocessing processes for CPU-bound tasks. For systems utilizing asyncio, any synchronous blocking call will halt the entire event loop, leading to denial-of-service and missed deadlines. Ruff's ASYNC rules strictly forbid blocking operations inside async functions, specifically utilizing synchronous HTTP clients (ASYNC210), opening files without async wrappers (ASYNC230), and employing time.sleep (ASYNC251).¹²

Industry Standard Documentation as Code Infrastructure

In highly regulated, safety-critical environments, documentation is not merely a supplementary developer convenience; it is a fundamental component of the software's structural integrity. Clear documentation drastically reduces the cognitive load required during security audits, accelerates project handovers, and enables rapid diagnostics during incident response scenarios.²⁷ Code must be documented under the philosophy that poor documentation quietly undermines operational stability.²⁸

The foundational standard for Python documentation is defined by PEP 257 (Docstring Conventions). It dictates the use of triple-double quotes ("""") for all docstrings, mandating that the docstring occur as the very first statement within a module, class, and function.¹⁷

For enterprise and safety-critical software, the explicitly mandated documentation standard is the **Google Style**. Google Style is exclusively required for its high legibility, low-boilerplate structure, and native parsing by language servers.³⁰

Under this mandated standard, a compliant Google Style docstring must exhaustively detail not only the expected inputs and outputs but explicitly declare the *exceptions raised* (using the `Raises: block`), the *side effects* produced, and any hardware/network restrictions regarding when the function can be safely called.³² Crucially, static type hinting must never replace the

docstring; type hints enforce the contract mechanically, while the docstring provides the semantic context, the underlying assumptions, and the rationale behind edge-case handling.¹⁴

Appendix A: The GenAI Architectural Extension

As Python serves as the undisputed lingua franca for Artificial Intelligence, deploying AI models within safety-critical perimeters introduces novel, highly complex attack vectors that traditional static analysis and memory safety rules cannot intercept. Machine learning models and Large Language Models (LLMs) operate probabilistically. They fail in non-deterministic ways and are highly susceptible to adversarial manipulation, data poisoning, and contextual hijacking.³³ Securing these systems requires integrating specialized governance frameworks and deploying defensive runtime controls.

Governing AI Risks with OWASP Frameworks

The foundational blueprint for identifying and mitigating AI vulnerabilities is the OWASP Top 10 for LLM Applications (updated for 2025).³⁴ Developers must actively architect defenses against these primary, critical threats:

- **LLM01: Prompt Injection:** The manipulation of LLMs via crafted inputs designed to bypass safety filters and override system instructions.³⁴ Attackers utilize complex taxonomies—specifically the Arcanum Prompt Injection Taxonomy—to deploy narrative framing, structural obfuscation (specifically base64 encoding and invisible Unicode), and logic inversion.³⁵ Systems must treat all external prompt data as highly untrusted, as attackers can embed invisible instructions to exfiltrate data and hijack the model's intent.
- **LLM02: Insecure Output Handling:** The dangerous practice of treating LLM output as safe, executable code.³⁴ Because LLMs can hallucinate and be manipulated into generating malicious payloads, their outputs must be rigorously sanitized and structurally validated before being passed to downstream execution layers and system shells.
- **LLM03 & LLM05: Data Poisoning and Supply Chain Vulnerabilities:** Relying on compromised upstream datasets and foundational models.³⁴ Poisoning involves attackers subtly modifying training data to introduce backdoors and bias into the model's logic, fundamentally compromising its reliability.

To systematically manage supply chain risk, organizations must implement an AI Bill of Materials (AIBOM). Utilizing open-source tools including the OWASP AIBOM Generator enables engineering teams to track the exact provenance, dependencies, and cryptographic hashes of every agent, model, and dataset deployed within the infrastructure.³⁵

Furthermore, the model artifacts themselves pose severe computational risks. Python's native serialization format, pickle, is inherently unsafe, as it allows for arbitrary remote code execution upon deserialization.³⁵ Advanced threat actors can embed malware directly within pre-trained model weights. To proactively close this vector, this standard **explicitly mandates the use of the safetensors format** for all saved machine learning models. safetensors stores tensor data

purely mathematically without executable code. PyTorch's default .pt and .pth extensions are strictly forbidden. Even with this secure format, developers must strictly employ the **modelscan** static analysis tool to inspect model artifacts for hidden execution triggers before loading them into operational memory.³⁵

Defending Agentic AI and the Model Context Protocol (MCP)

Agentic AI systems—where LLMs are granted autonomy to execute tools, perform network requests, and dynamically modify their environment—exponentially expand the attack surface.³⁷ The principle of "Excessive Agency" (LLM08) warns that granting models unchecked autonomy leads to catastrophic, unintended consequences that jeopardize system reliability and user trust.³⁴

When autonomous agents utilize architectures including the Model Context Protocol (MCP) to seamlessly interface with external third-party tools and data silos, the system architecture must enforce absolute trust boundaries. Best practices for securing Agentic AI dictate:

1. **Trajectory-Level Risk Assessment:** Security can no longer be evaluated based on a single prompt and single output. Frameworks must analyze the multi-step "trajectory" of an agent's execution to ensure that a sequence of seemingly benign actions does not culminate in a critical security breach.³⁵
2. **Strict Sandboxing and Runtime Isolation:** AI agents must never execute code and interact directly with the host machine's filesystem and environment variables. All operations must be hermetically confined within ephemeral, highly restricted sandboxes using **gVisor**. gVisor provides hardware-enforced isolation.³⁵ Network egress must be heavily constrained to prevent agents from establishing command-and-control communication and exfiltrating sensitive context.
3. **Secure MCP Gateways:** The integration of third-party MCP servers requires intermediate middleware gateways.³⁵ These secure gateways handle cryptographic authentication, restrict tool discovery to an explicit organizational allowlist, and perform real-time monitoring of all agent-tool interactions. This prevents "line jumping" attacks and blocks malicious changes to server configurations.³⁵
4. **Human-in-the-Loop (HITL) Oversight:** High-impact, irreversible actions—including modifying production databases, authorizing financial transactions, and dispatching external communications—must be decoupled from the agent's autonomy. These actions require explicit, out-of-band human authorization, acting as a final, un-hackable circuit breaker against hallucinated objectives and successful prompt injections.³⁷

Implementing Input and Output Guardrails

The highly dynamic, probabilistic nature of Generative AI means that traditional static type checking and linters cannot evaluate the semantic safety of a generated string. Therefore, the implementation of programmable, active guardrails is an architectural mandate.

The **NeMo-Guardrails** framework operates as a Generative Application Firewall (GAF).³⁵ This

toolkit sits as an intermediary layer between the user, the LLM, and the downstream application infrastructure.

On the input side, these guardrails scan incoming requests against known prompt injection pattern databases. They actively detect Personally Identifiable Information (PII), applying masking and redaction to prevent accidental data leakage into the model's context window, and block toxic and adversarial framing attempts.³⁵ On the output side, guardrails verify factual consistency to mitigate hallucinations, enforce strict structural constraints (specifically ensuring a JSON payload is valid before parsing), and ensure the model has not generated malicious code and hallucinated API parameters.³⁵

By aggressively offloading access control, formatting enforcement, and safety filtering away from the LLM's internal system prompt—which is easily bypassed by skilled adversaries—and into an external, deterministic Python execution layer, developers restore a critical degree of architectural reliability to AI systems.

Conclusion

Adapting Python for safety-critical systems requires a rigorous, systematic departure from the language's inherently dynamic, permissive, and dynamically-allocated culture. By translating the NASA/JPL Power of Ten rules into highly specific Pythonic constraints, developers can achieve necessary structural determinism. This is realized by strictly prohibiting recursive complexity, enforcing `__slots__` and pre-allocation for memory predictability, utilizing deep immutable data topologies to prevent reference aliasing, and entirely eliminating the use of dynamic metaprogramming.

Crucially, the enforcement of these constraints is no longer relegated to subjective manual code review. It is computationally guaranteed through the integration of modern, high-performance, Rust-backed tooling. The implementation of uv universal lockfiles ensures absolute, bit-for-bit environmental reproducibility across all deployment stages. Simultaneously, Ruff acts as an uncompromising, zero-tolerance gatekeeper against excessive cyclomatic complexity, unhandled exceptions, and insecure code patterns, while Semgrep provides deep taint analysis. Coupled with standardized, structured documentation utilizing Google docstring conventions, the codebase transforms into a highly analyzable, self-documenting architecture.

Finally, as Python continues to power the integration of Large Language Models and autonomous agentic workflows, traditional software assurance methodologies must be expanded to counter AI-specific threats. By actively adopting OWASP GenAI guidelines, tracking model provenance via AI Bills of Materials (AIBOMs), aggressively sandboxing agentic execution runtimes using gVisor, and deploying NeMo-Guardrails, organizations can safely leverage the advanced, probabilistic capabilities of modern AI while maintaining the hardened, fault-tolerant integrity required of life-critical and mission-critical systems.

Works cited

1. P10.pdf
2. The Power of 10: Rules for Developing Safety-Critical Code - Wikipedia, accessed March 25, 2026,
https://en.wikipedia.org/wiki/The_Power_of_10:_Rules_for_Developing_Safety-Critical_Code
3. Assessing Python's suitability for airborne safety-critical systems under DO-178C guidelines, accessed March 25, 2026,
<https://research-repository.griffith.edu.au/items/f2541b4b-5734-4852-9630-314d9fb6c35d>
4. Why Python Struggles in Safety-Critical Embedded Systems - Safelink Innovations, accessed March 25, 2026,
<https://www.safelink-innovations.com/post/python-in-embedded-programming>
5. NASA's 10 Coding Rules Explained: How to Build Reliable and Safe Software, accessed March 25, 2026,
<https://www.aikido.dev/code-quality/rules/nasa-10-coding-rules-for-safety-critical-code>
6. deep-research-report.md
7. Memory Management — Python 3.14.3 documentation, accessed March 25, 2026,
<https://docs.python.org/3/c-api/memory.html>
8. Understanding Memory Management in Python: A Beginner's Guide | by Sunil Nepali, accessed March 25, 2026,
<https://medium.com/@sunilnepali844/understanding-memory-management-in-python-a-beginners-guide-e440769e4275>
9. How to Create Memory-Efficient Classes with `__slots__` - OneUptime, accessed March 25, 2026,
<https://oneuptime.com/blog/post/2026-01-30-how-to-create-memory-efficient-classes-with-slots/view>
10. Deep Dive into Slots Optimizing Python Class Memory Usage - Leapcell, accessed March 25, 2026,
<https://leapcell.io/blog/deep-dive-into-slots-optimizing-python-class-memory-usage>
11. Python for High Performance - Writing Faster Python - Memory Management, accessed March 25, 2026,
<https://cvw.cac.cornell.edu/python-performance/faster-python/memory-management>
12. Rules | Ruff - Astral Docs, accessed March 25, 2026,
<https://docs.astral.sh/ruff/rules/>
13. too-many-statements (PLR0915) | Ruff - Astral Docs, accessed March 25, 2026,
<https://docs.astral.sh/ruff/rules/too-many-statements/>
14. docstrings | Python Best Practices, accessed March 25, 2026,
<https://realpython.com/ref/best-practices/docstrings/>
15. Ruff users, what rules are using and what are you ignoring? : r/Python - Reddit, accessed March 25, 2026,

- https://www.reddit.com/r/Python/comments/1kttfst/ruff_users_what_rules_are_using_and_what_are_you/
16. Python Typing in 2025: A Comprehensive Guide | by Khaled Jallouli - Medium, accessed March 25, 2026, <https://khaled-jallouli.medium.com/python-typing-in-2025-a-comprehensive-guide-d61b4f562b99>
 17. Google Python Style Guide, accessed March 25, 2026, <https://google.github.io/styleguide/pyguide.html>
 18. The Concept of Pointers in C and C++ and Why Python Doesn't Have Them - Medium, accessed March 25, 2026, <https://medium.com/@abhishekjainindore24/the-concept-of-pointers-in-c-and-c-and-why-python-doesnt-have-them-9ade1875d9ed>
 19. State of the Art Python in 2024 - Reddit, accessed March 25, 2026, https://www.reddit.com/r/Python/comments/1ghiln0/state_of_the_art_python_in_2024/
 20. Using uv: A Modern Python Workflow | Tyler Crosse, accessed March 25, 2026, <https://www.tylercrosse.com/ideas/2025/uv/>
 21. Managing Python Environments: pyenv and uv Tutorial (Data Science Engineering Gap Part 1) - Crow Intelligence, accessed March 25, 2026, <https://crowintelligence.org/2025/10/09/managing-python-environments-pyenv-and-uv-tutorial-data-science-engineering-gap-part-1/>
 22. uv - Astral Docs, accessed March 25, 2026, <https://docs.astral.sh/uv/>
 23. UV and Ruff: Turbocharging Python Development with Rust-Powered Tools, accessed March 25, 2026, <https://www.devtoolsacademy.com/blog/uv-and-ruff-turbocharging-python-development-with-rust-powered-tools/>
 24. How to use a uv lockfile for reproducible Python environments, accessed March 25, 2026, <https://pydevtools.com/handbook/how-to/how-to-use-a-uv-lockfile-for-reproducible-python-environments/>
 25. astral-sh/uv: An extremely fast Python package and project manager, written in Rust. - GitHub, accessed March 25, 2026, <https://github.com/astral-sh/uv>
 26. GitHub - astral-sh/ruff: An extremely fast Python linter and code formatter, written in Rust., accessed March 25, 2026, <https://github.com/astral-sh/ruff>
 27. Code Documentation Best Practices and Standards: A Complete Guide - Codacy | Blog, accessed March 25, 2026, <https://blog.codacy.com/code-documentation>
 28. How to Write Technical Documentation in 2025: A Step-by-Step Guide - DEV Community, accessed March 25, 2026, <https://dev.to/auden/how-to-write-technical-documentation-in-2025-a-step-by-step-guide-1hh1>
 29. PEP 8 – Style Guide for Python Code, accessed March 25, 2026, <https://peps.python.org/pep-0008/>
 30. What are the most common Python docstring formats? - Codemia, accessed March 25, 2026, https://codemia.io/knowledge-hub/path/what_are_the_most_common_python_d

[ocstring_formats](#)

31. sphinx.ext.napoleon – Support for NumPy and Google style docstrings, accessed March 25, 2026,
<https://www.sphinx-doc.org/en/master/usage/extensions/napoleon.html>
32. Documenting Python Code: A Complete Guide, accessed March 25, 2026,
<https://realpython.com/documenting-python-code/>
33. OWASP AI Testing Guide, accessed March 25, 2026,
<https://owasp.org/www-project-ai-testing-guide/>
34. OWASP Top 10 for Large Language Model Applications, accessed March 25, 2026,
<https://owasp.org/www-project-top-10-for-large-language-model-applications/>
35. ottosulin/awesome-ai-security: A collection of awesome ... - GitHub, accessed March 25, 2026, <https://github.com/ottosulin/awesome-ai-security>
36. Securing Agentic Applications Guide 1.0 - OWASP Gen AI Security ..., accessed March 25, 2026,
<https://genai.owasp.org/resource/securing-agentic-applications-guide-1-0/>
37. AI Agent Security - OWASP Cheat Sheet Series, accessed March 25, 2026,
https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html